

1. What is a Module in Python

A **module** is simply a **Python file (.py)** that contains **variables, functions, and classes**. Modules help you **organize and reuse code** in different Python programs.

Benefits of using modules:

- Organizes code logically
 - Encourages reusability
 - Easier maintenance and debugging
-

Example: Creating and Using a Module

Step 1 — Create a module file

File: math_utils.py

```
# math_utils.py
```

```
# variable
```

```
pi = 3.14159
```

```
# function
```

```
def area_of_circle(radius):  
    return pi * (radius ** 2)
```

```
# class
```

```
class Calculator:  
    def add(self, a, b):  
        return a + b
```

Step 2 — Use this module in another file

File: main.py

```
# main.py
```

```
# import entire module
```

```
import math_utils
```

```
print("Pi value:", math_utils.pi)

print("Area of circle:", math_utils.area_of_circle(5))
```

```
calc = math_utils.Calculator()

print("Sum:", calc.add(10, 20))
```

Output

Pi value: 3.14159

Area of circle: 78.53975

Sum: 30

Different Ways to Import a Module

1. Import entire module

```
import math_utils
```

2. Import specific items

```
from math_utils import area_of_circle, pi
```

3. Import with alias

```
import math_utils as mu
```

4. Import all items (not recommended)

```
from math_utils import *
```

2. What is a Package in Python

A **package** is a **collection of related modules** stored in a **directory** that contains a special file called `__init__.py`.

Packages allow you to **organize large projects into smaller modules** and create **hierarchical module structures**.

Benefits of using packages:

- Organizes related code together
- Prevents naming conflicts
- Allows nested imports

Directory Structure of a Package

```
my_package/
|
├── __init__.py
├── math_utils.py
├── string_utils.py
└── sub_package/
    ├── __init__.py
    └── advanced_math.py
```

Example: Using a Package

my_package/math_utils.py

```
# math_utils.py
```

```
def square(n):
```

```
    return n ** 2
```

my_package/string_utils.py

```
# string_utils.py
```

```
def reverse_string(s):
```

```
    return s[::-1]
```

my_package/sub_package/advanced_math.py

```
# advanced_math.py
```

```
def cube(n):
```

```
    return n ** 3
```

main.py

```
# main.py
```

```
# Import from top-level package
```

```
from my_package import math_utils, string_utils
```

```
print(math_utils.square(4))    # 16
```

```
print(string_utils.reverse_string("Python")) # nohtyP
```

```
# Import from sub-package
```

```
from my_package.sub_package import advanced_math
```

```
print(advanced_math.cube(3))      # 27
```

3. Role of `__init__.py`

- Tells Python that the directory should be treated as a package.
- Can be empty or used to initialize package-level imports or variables.

Example `__init__.py`:

```
# my_package/__init__.py
```

```
print("Initializing my_package")
```

```
# Optional: pre-import commonly used modules
```

```
from .math_utils import square
```

```
from .string_utils import reverse_string
```

Then you can do:

```
from my_package import square, reverse_string
```

4. Using Built-in (Standard Library) Modules

Python comes with many built-in modules. Example:

```
import math
```

```
import random
```

```
import datetime
```

```
print(math.sqrt(16))
```

```
print(random.randint(1, 10))
```

```
print(datetime.date.today())
```

5. Summary Table

Concept	Description	Example
Module	A single .py file with code	math_utils.py
Package	A folder with __init__.py and multiple module files	my_package/
Importing	Use import or from ... import ...	from my_package import math_utils
Purpose	Organize and reuse code	Break down big project into smaller parts
